

Exercícios – Revisão para P1

1) Arquitetura de software é uma representação que permite analisar a efetividade do projeto no entendimento dos requisitos declarados. Durante a fase de concepção da arquitetura, podem-se considerar alternativas de arquitetura em um estágio em que mudanças ainda são realizadas com menor esforço, diminuindo riscos associados à construção do software ainda na fase inicial. (PRESSMAN, R. S. Engenharia de Software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016 (adaptado)).

A respeito dos estilos e padrões arquiteturais contidos na engenharia de software, avalie as afirmações a seguir.

I. Em arquiteturas orientadas a objetos, a comunicação entre os componentes do software é realizada por intermédio da troca de mensagens.

II. As arquiteturas monolíticas consistem de um sistema dividido em pequenas partes, possibilitando que estas tenham sua manutenção, execução e evolução individual.

III. No padrão arquitetural Modelo-Visão-Controle (MVC), a camada de Modelo representa dados associados às interações realizadas no Controle, podendo ser apresentados/manipulados posteriormente na Visão.

IV. Nas arquiteturas de microsserviços, o software possui componentes altamente acoplados, dificultando a manutenção.

É correto apenas o que se afirma em:

- a) I e III.
- b) II e III.
- c) II e IV.
- d) I, II e IV.
- e) I, III e IV.

2) A tendência de desenvolver aplicações que possam ser utilizadas em nuvem otimiza o desempenho de máquinas locais, munidas de poucos recursos computacionais, em uma rede e garante a possibilidade desses computadores utilizarem outros recursos computacionais de maior desempenho, bem como a inserção de novos hosts neste ambiente computacional, sem comprometimento do sistema como um todo.

Esse cenário pressupõe a utilização de uma arquitetura:

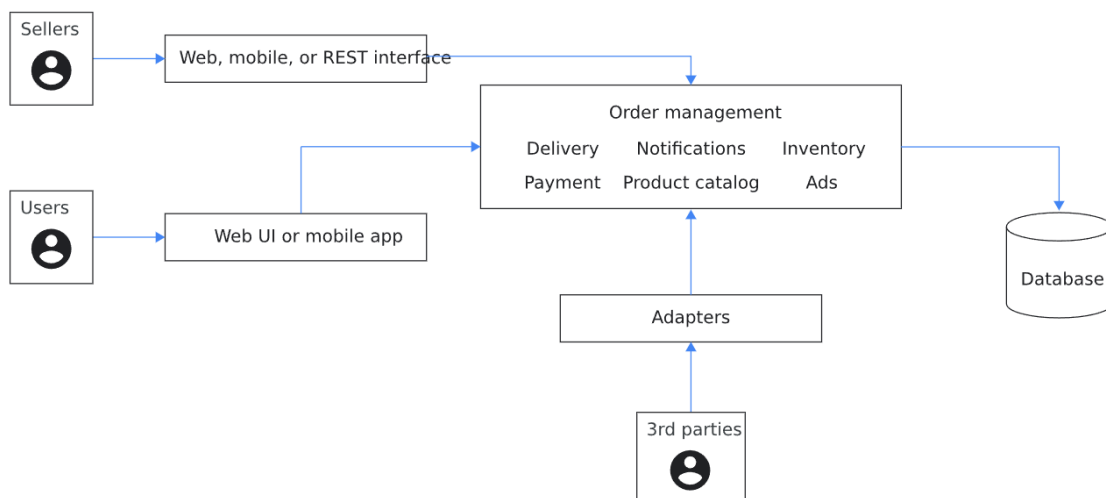
- a) monolítica

- b) em camadas
- c) monocamada
- d) cliente-servidor
- e) de máquina virtual

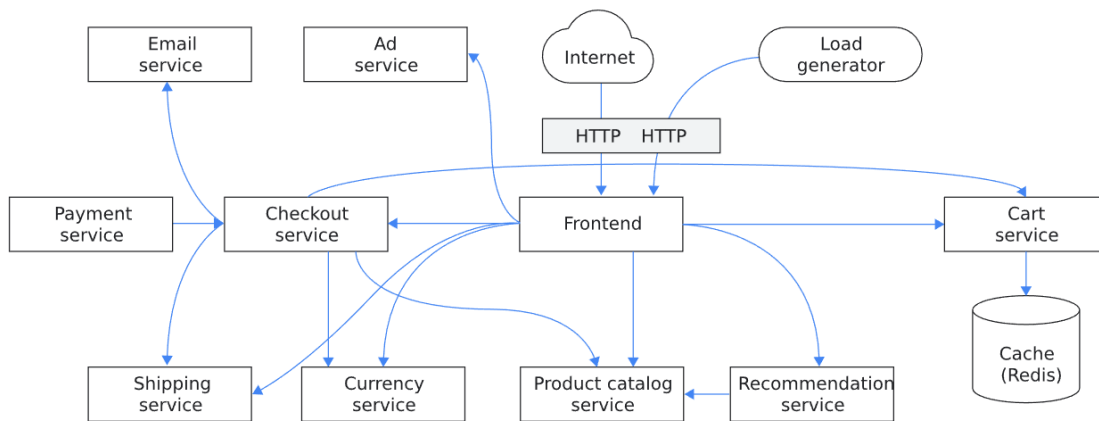
3) Acerca da arquitetura de microsserviços, assinale a opção correta.

- a) A arquitetura de microsserviços é um padrão para a criação de aplicações distribuídas, porém não possui alta escalabilidade.
- b) A comunicação entre os microsserviços é feita por meio de mecanismos padrões de tecnologia, como, por exemplo, o REST (representational state transfer).
- c) Microsserviços utilizam uma única base de dados lógica para a persistência de dados.
- d) Um requisito fundamental da arquitetura de microsserviços é o uso de versionamento de mudanças.
- e) Os microsserviços são componentes autônomos e de alto acoplamento, de modo que há a necessidade de se utilizar uma mesma linguagem na sua construção.

4) Um aplicativo monolítico é um aplicativo de software de nível único em que módulos diferentes são combinados em um único programa. O diagrama a seguir mostra um exemplo de configuração de aplicativo de comércio eletrônico, em que o aplicativo consiste em vários módulos. Os diferentes módulos no aplicativo de comércio eletrônico correspondem à lógica de negócios para pagamento, entrega e gerenciamento de pedidos. Todos esses módulos são empacotados e implantados como um único executável lógico. O formato real depende da linguagem e do framework do aplicativo. Por exemplo, muitos aplicativos Java são empacotados como arquivos JAR e implantados em servidores de aplicativos tal como o Tomcat.



No diagrama a seguir, apresenta-se uma possível decomposição do aplicativo de comércio eletrônico em microsserviços:



Com base no que foi exposto, responda:

- a) Aponte dois benefícios e dois desafios do uso de monolitos.
- b) Apronte dois benefícios e dois desafios do uso de microserviços.

5) Usualmente, os aplicativos monolíticos têm bancos de dados monolíticos. Um dos princípios de uma arquitetura de microserviços é ter um banco de dados para cada microserviço. Portanto, ao modernizar o aplicativo monolítico para microserviços, você precisa dividir o banco de dados monolítico com base nos limites do serviço identificados. No entanto, dividir um banco de dados monolítico é complexo porque pode não haver uma separação clara entre os objetos do banco de dados. Além desse desafio, quais outros problemas podem ser enfrentados em relação à distribuição de bases de dados em sistema de microserviços?

GABARITO:

1) A

2) D

3) B

4)

a)

Benefícios:

É possível implementar testes completos de um aplicativo monolítico usando ferramentas como o Selenium.

Para implantar um aplicativo monolítico, basta copiá-lo para um servidor.

Todos os módulos em um aplicativo monolítico compartilham memória, espaço e recursos. Assim, é possível usar uma única solução para lidar com questões transversais, como geração de logs, armazenamento em cache e segurança.

A abordagem monolítica pode fornecer vantagens de desempenho, porque os módulos podem chamar uns aos outros diretamente. Por outro lado, os microsserviços normalmente exigem uma chamada de rede para se comunicar entre si.

Desafios:

Os aplicativos geralmente crescem com o tempo. Pode se tornar complicado implementar alterações em um aplicativo grande e complexo que tem módulos estreitamente acoplados. Como qualquer alteração de código afeta todo o sistema, você precisa coordenar completamente as alterações. A coordenação de alterações torna o processo geral de desenvolvimento e teste muito mais longo em comparação com aplicativos de microsserviço.

Pode ser complicado conseguir integração e implantação contínuas (CI/CD) com um sistema monolítico. Essa complexidade ocorre porque é necessário reimplantar todo o aplicativo para atualizar qualquer parte dele. Além disso, é provável que você precise fazer testes manuais abrangentes de todo o aplicativo para verificar regressões.

Os aplicativos monolíticos podem ser difíceis de escalonar quando diferentes módulos têm requisitos de recursos conflitantes. Por exemplo, um módulo pode implementar uma lógica de processamento de imagem com uso intensivo de CPU. Outro módulo pode ser um banco de dados na memória. Como esses módulos são implantados juntos, você precisa comprometer a escolha do hardware.

Como todos os módulos são executados no mesmo processo, um bug em qualquer módulo, como um vazamento de memória, pode derrubar todo o sistema.

Os aplicativos monolíticos aumentam a complexidade quando você quer adotar novos frameworks e linguagens. Por exemplo, é caro (em tempo e dinheiro) reescrever um aplicativo inteiro para usar uma nova estrutura, mesmo que essa estrutura seja consideravelmente melhor.

b)

Benefícios:

A funcionalidade total não foi alterada, mas você usa microsserviços para separar o aplicativo em blocos ou serviços gerenciáveis. Cada serviço tem um limite bem definido na forma de uma API RPC ou orientada por mensagens. Portanto, os serviços individuais podem ser mais rápidos de desenvolver, mais fáceis de entender e manter.

As equipes autônomas podem desenvolver serviços individuais de forma independente. É possível organizar microsserviços em torno dos limites de negócios, e não os recursos técnicos de um produto. Você organiza suas equipes para uma responsabilidade única e independente para todo o ciclo de vida de seu software atribuído, desde o desenvolvimento até o teste, a implantação e a manutenção e monitoramento.

O processo de desenvolvimento de microsserviços independente também permite que os desenvolvedores escrevam cada microsserviço em uma linguagem de programação diferente, criando um aplicativo poliglota. Ao usar a linguagem mais eficaz para cada microsserviço, você pode desenvolver um aplicativo mais rapidamente e otimizá-lo para reduzir a complexidade do código e aumentar o desempenho e a funcionalidade.

Ao desacoplar recursos de um sistema monolítico, é possível fazer com que as equipes independentes liberem o microsserviço de modo independente. Ciclos de lançamento independentes podem ajudar a melhorar a velocidade e o tempo de produção das suas equipes no mercado.

A arquitetura de microsserviços também permite escalonar cada serviço de maneira independente. É possível implantar o número de instâncias de cada serviço que atendam às restrições de capacidade e disponibilidade. Também é possível usar o hardware que melhor atende aos requisitos de recursos de um serviço. Ao escalonar serviços de maneira independente, você ajuda a aumentar a disponibilidade e a confiabilidade de todo o sistema.

Desafios:

Um grande desafio dos microsserviços é a complexidade causada porque o aplicativo é um sistema distribuído. Os desenvolvedores precisam escolher e implementar um mecanismo de comunicação entre serviços. Os serviços também precisam lidar com falhas parciais e indisponibilidade dos serviços dependentes.

Outro desafio com microsserviços é que você precisa gerenciar transações em diferentes microsserviços (também chamados de transação distribuída). As operações de negócios que atualizam várias entidades de negócios são bastante comuns e geralmente são aplicadas de maneira atômica, em que todas as operações são aplicadas ou tudo falha. Ao unir várias operações em uma única transação do banco de dados, você garante a atomicidade. Em um aplicativo baseado em microsserviços, as operações de negócios podem estar espalhadas por microsserviços diferentes. Portanto, você precisa atualizar vários bancos de dados de propriedade de serviços diferentes. Se houver uma falha, não é trivial rastrear a falha ou o sucesso de chamadas para os diferentes microsserviços e o estado de reversão. O pior cenário pode resultar em dados inconsistentes entre os serviços quando a reversão do estado devido a falhas não aconteceu corretamente.

O teste abrangente de aplicativos baseados em microsserviços é mais complexo do que testar um aplicativo monolítico. Por exemplo, para testar a funcionalidade de processamento de um pedido em um serviço de comércio eletrônico monolítico, selecione os itens, adicione-os a um carrinho e faça check-out. Para testar o mesmo fluxo em uma arquitetura baseada em microsserviços, vários serviços, como front-end, pedido e pagamento, chamam uns aos outros para concluir a execução do teste.

A implantação de um aplicativo baseado em microsserviços é mais complexa do que a implantação de um aplicativo monolítico. Um aplicativo de microsserviço geralmente consiste em muitos serviços, cada um com várias instâncias de ambiente de execução. Você também precisa implementar um mecanismo de descoberta de serviço que permita que um serviço descubra os locais de outros serviços com os quais ele precisa se comunicar.

Uma arquitetura de microsserviços adiciona sobrecarga de operações porque há mais serviços para monitorar e alertar. A arquitetura de microsserviços também tem mais pontos de falha devido ao aumento dos pontos de comunicação de serviço a serviço. Um aplicativo monolítico pode ser implantado em um pequeno cluster de servidor de aplicativos. Um aplicativo baseado em microsserviços pode ter dezenas de serviços separados para criar, testar, implantar e executar, potencialmente em várias linguagens e ambientes. Todos esses serviços precisam ser agrupados para failover e resiliência. A produção de um aplicativo de microsserviços requer infraestrutura de operações e monitoramento de alta qualidade.

A divisão de serviços em uma arquitetura de microsserviço permite que o aplicativo execute mais funções ao mesmo tempo. No entanto, como os módulos são executados como serviços isolados, a latência é introduzida no tempo de resposta devido a chamadas de rede entre serviços.

Nem todos os aplicativos são grandes o suficiente para serem divididos em microsserviços. Além disso, alguns aplicativos exigem uma forte integração entre componentes. Por exemplo, aplicativos que precisam processar fluxos rápidos de dados em tempo real. Qualquer camada extra de comunicação entre serviços pode retardar o processamento em tempo real. Pensar na comunicação entre serviços com antecedência pode trazer insights úteis para marcar claramente os limites dos serviços.

5) Você também precisa considerar outros problemas, como sincronização de dados, integridade transacional, mesclagens e latência.